

Queue Cure 2026

Supporting Documentation

Real-Time Clinic Queue Management System

Flask • Socket.IO • SQLite • Bootstrap 5

Hackathon Submission

Table of Contents

- [Socket Event Diagram](#)
- [Thought Process](#)
- [API Documentation](#)
- [Test Cases](#)
- [Deployment Guide](#)

Socket Event Diagram

Real-Time Communication Flow

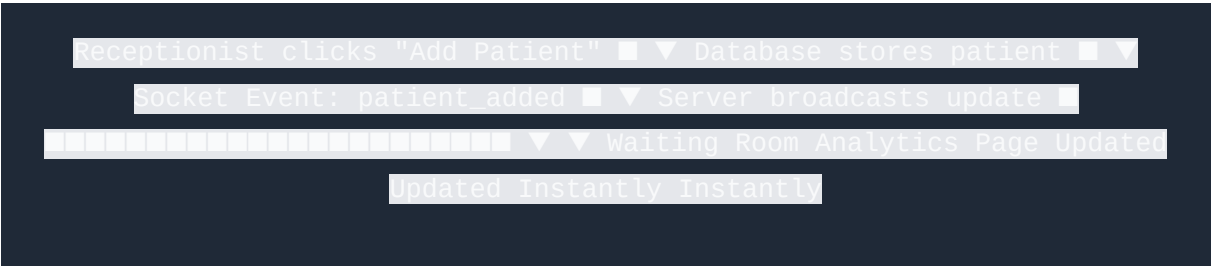


Socket Events

Event	Trigger	Purpose
patient_added	New patient added	Update all screens
patient_removed	Patient removed	Refresh queue
patient_called	Next token called	Update current token
queue_updated	Queue state changes	Synchronize clients

settings_updated	Consultation time changed	Recalculate wait times
queue_reset	Queue cleared	Reset all screens
analytics_updated	Analytics recalculated	Refresh metrics
request_state	Client reconnects	Recover latest state

Example Flow: Patient Added



Why WebSockets?

Traditional Polling

- Client → Request → Server → Response → Repeat
- Problems:
- Delayed updates
- Higher server load
- Poor user experience

Queue Cure 2026 WebSockets

- Server ↔ Client
 - Benefits:
 - Instant updates
 - Lower bandwidth
 - Better scalability
 - Improved patient experience
-

Thought Process

Problem Statement

Many clinics still rely on paper-token systems, manual announcements, and disconnected communication between receptionists and patients, leading to:

- Patients unaware of their position or wait time
- Receptionists repeating status updates verbally
- No visibility into daily throughput or analytics
- Disconnected experience between staff and waiting patients

Why This Problem Matters

- Paper tokens are inefficient and error-prone
- Manual announcements cause frustration and confusion
- Lack of real-time data hurts clinic operations
- Poor patient experience affects satisfaction

Solution Overview

Queue Cure 2026 is a modern real-time healthcare queue management system that:

- Provides live queue visibility
- Eliminates manual announcements
- Calculates wait times automatically
- Tracks doctor status
- Generates operational analytics
- Uses WebSockets for instant updates

Architecture Decisions

Component	Choice	Reason
Backend	Flask	Lightweight, easy to deploy, great for hackathons
Real-Time	Flask-SocketIO	Native WebSocket support for instant updates
Database	SQLite	File-based, no server needed, perfect for small clinics
Frontend	Bootstrap 5	Responsive, professional, fast to build
Deployment	Render	Free tier, auto-config from render.yaml, persistent disk

Why Flask-SocketIO Was Chosen

- Seamless integration with Flask
- Automatic fallback mechanisms
- Easy to broadcast events to multiple clients

- Works great with gunicorn + eventlet
- Perfect for small to medium clinic deployments

Real-Time Communication Strategy

- Every user action on the dashboard triggers a Socket.IO event
- Server updates the database then broadcasts the event
- All connected clients receive the update
- No polling, no manual refresh

Concurrency Handling

- SQLite serializes writes, preventing race conditions
- Flask-SocketIO + eventlet worker supports multiple concurrent connections
- Multi-tab updates work perfectly via broadcast events

Edge Cases Handled

- Empty queue: Call Next shows error, no patient displayed
- Duplicate patient names: 409 Conflict response, patient not added
- Invalid consultation time: Validates between 1-120 minutes
- Multiple tabs: All tabs update simultaneously via Socket.IO
- Browser reconnect: Client sends request_state event to recover latest data
- Simultaneous updates: Server handles in order via SQLite serialization

Scalability Considerations

Current setup perfect for small/medium clinics; for larger clinics:

- Switch to PostgreSQL
- Use Redis for Socket.IO message broker
- Add load balancing
- Implement user authentication

Future Improvements

- SMS/WhatsApp notifications when patient's turn is approaching
 - Multi-doctor/multi-room support
 - Appointment scheduling integration
 - QR code check-in for patients
 - Historical analytics with date range filters
 - PWA support for mobile receptionist app
 - Role-based access control
-

API Documentation

Patients API

Method	Endpoint	Purpose
GET	/api/patients	Get all patients (active queue + history)
POST	/api/patients	Add new patient
POST	/api/patients/call-next	Call next patient in queue
POST	/api/patients/:id/call	Call specific patient
DELETE	/api/patients/:id	Remove patient from queue
POST	/api/patients/:id/complete	Mark patient's consultation as complete

Settings API

Method	Endpoint	Purpose
GET	/api/settings	Get clinic settings
PUT	/api/settings	Update clinic settings

Queue API

Method	Endpoint	Purpose
POST	/api/queue/reset	Reset active queue (keep history)

Analytics API

Method	Endpoint	Purpose
GET	/api/analytics	Get analytics metrics

GET	/api/export	Export patient history as CSV
-----	-------------	----------------------------------

Test Cases

Functional Tests

Test Case	Expected Result	Status
Add Patient	Patient added, token assigned, all screens update	■ Passing
Remove Patient	Patient removed, queue updates	■ Passing
Call Next Token	Patient marked as called, current token updates	■ Passing
Queue Reset	Queue cleared, screens reset	■ Passing
Change Consultation Time	Time saved, wait times recalculated	■ Passing

Edge Cases

Test Case	Expected Result	Status
Empty Queue Call Next	Error message: "Queue is empty"	■ Passing
Duplicate Patient Names	409 Conflict: Patient already in queue	■ Passing
Invalid Consultation Time	Validation error	■ Passing
Multiple Tabs Open	Second tab updates instantly	■ Passing
Browser Reconnect	Page connects and recovers latest state	■ Passing
Simultaneous Updates	Both tabs update, no duplicate tokens	■ Passing

Deployment Guide

Local Setup

Clone repo:

```
bash git clone https://github.com/VanitaCSE/queue-cure-2026.git cd
queue-cure-2026/queue-cure
```

Create virtual environment:

```
``bash
```

Windows

```
python -m venv venv
```

```
.\venv\Scripts\activate
```

macOS/Linux

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
``
```

Install requirements:

```
bash pip install -r requirements.txt
```

Run Flask app:

```
bash python app.py
```

Open in browser:

4. Landing: <http://localhost:5000>
5. Dashboard: <http://localhost:5000/dashboard>
6. Waiting Room: <http://localhost:5000/patient>
7. Analytics: <http://localhost:5000/analytics>

GitHub Push Process

1. Make changes locally

2. Stage, commit, push:

```
bash git add . git commit -m "Your commit message" git push origin  
main
```

Render Deployment Process

1. Log in to <https://render.com>

2. Click "New" → "Web Service"

3. Select your GitHub repo: VanitaCSE/queue-cure-2026

4. Configure (render.yaml auto-fills):

5. Name: queue-cure-2026

6. Runtime: Python 3

7. Branch: main

8. Build Command: pip install -r requirements.txt

9. Start Command: gunicorn --worker-class eventlet -w 1 --bind 0.0.0.0:\$PORT
app:app

10. Plan: Free

11. Click "Create Web Service"

Environment Variables

Key	Value
-----	-------

PYTHON_VERSION	3.11.9
SECRET_KEY	Auto-generated by Render
DATABASE_PATH	/var/data/database.db

Verification Checklist

- ☐ App loads without errors
- ☐ Dashboard accessible
- ☐ Patient can be added
- ☐ Real-time updates work
- ☐ Waiting Room updates
- ☐ Analytics dashboard loads
- ☐ CSV export works
- ☐ Dark mode toggle works
- ☐ Doctor status updates
- ☐ All tests pass locally
- ☐ No console errors